Special Participation C: HW10 Refactor

Nils Selte and Hanna Roed

We started by making the code in `architectures.py` clearer and easier to work with by adding type hints and short, readable docstrings. This file was already nicely organized in a class, so our goal was mainly to make it “less overwhelming” by making it easier to understand each part separately. The type hints make it clear what each function or method expects and returns, which helps students quickly understand the code. The docstrings of course explain what each class or method does and will then show once you hover over the function when you want to use it (in VSCode at least).

Our approach here follows Python’s guidelines in PEP 8 (style), PEP 257 (docstrings), and PEP 484 (type hints). The models now state their assumptions more clearly, such as ResNet18 expecting inputs resized to 224×224 and the MLP taking flattened 32×32×3 inputs. We also cleaned up a few small issues, such as removing a duplicate import.

We added a root README to ideally act as an entry point for students. The README explains the purpose of the project, the directory structure, how the refactored parts were improved, and how to quickly use the models in simple Python examples. Having this information at the top level makes it easier to get started and reduces confusion about where to look and what to run.

To make setup more (modern and) consistent across different machines, we set up for using a UV environment. With UV it is easy to create an isolated environment and install exactly what you need for the given repo. This helps reduce conflicts with system packages, and supports reproducible setups in a neat way. It also fits current trends in Python tooling that emphasize simple commands, predictable environments, and good performance.

We also added comments aimed for improving students' intuition to (most of) the code cells in the q_hand_transformer notebook. Building a transformer from scratch is difficult, and short, targeted comments clarify intent without being overwhelming. We also added small notes about how attention behaves with repeated tokens to give intuition about weighted sums and softmax distributions (it never hurts to have a refresher available while doing the homework). These comments are intentionally short and are an addition to the existing comments/text so they support, rather than replace, the original teaching flow.

To implement these changes we used GPT-5 in Copilot agent mode + us doing some minor correcting to the changes it made. The UV setup we did ourselves in the terminal. The prompts were as follows:

- Here is our task
  "Because the code in our problems was evolved till it worked with specific deep-learning-concept related learning objectives, it is often not good code from the perspective of being exemplary from a software engineering point of view. For example, it is often not very pythonic, etc. You can, ideally with AI assistance that you document carefully vis-a-vis process, take one of the coding problems/demos and refactor it as well as update the code to follow good documented software engineering and ML Engineering processes. Here, we expect you to give citations to the relevant points of good style and document your changes in a report. The constraint is that the problem code shouldn't lose any of its teaching value --- just be transformed to have good coding practices and style. As always, deconfliction is a must however this can be a group effort by up to three people. "
  The first thing we should start with is adding typehints and short understandable docstrings to the achitectures.py file.
- Add a simple README file in the root directory.
- In q_hand_transformer.ipynb can you, without changing any code or removing previous commenting, add comments where it is natural in the coding chunks that are short but explanatory, remember this should give intuition.

The final product and all changes can be seen in this repository:
https://github.com/hannaroed/CS282-hw10-refactor